WeCloudData

الأكاديمية السعودية الرقمية
SAUDI DIGITAL ACADEMY

PROJECT PROPOSAL



JobDataPipeline

SUBMITTED BY : TEAM C

MEMBERS :

GHALA ALSALEM
MURADI ALMUTAIRI
MOHAMMED ALHARBI
LAMIA ALSUHIBANI

1. OVERVIEW

The **Job Data Pipeline** project is designed to automate the collection and processing of technology job market data in Saudi Arabia from multiple sources. The pipeline will transform raw technology job postings into a structured and analysis-ready dataset by applying data cleaning, standardization, translation, and integration processes.

The resulting dataset will support the analysis of technology job demand, job locations, companies, job roles, and market trends across Saudi Arabia, providing reliable data for generating meaningful insights and supporting data-driven decisions about the Saudi technology job market.

2. PROBLEM STATEMENT

2.1 Current Situation

The technology job market generates a large volume of data across multiple sources. However, this information is often fragmented, inconsistent, and available in different formats and languages, making it difficult to obtain a clear and reliable view of current market demand.

2.2 Key Challenges

- Job data is distributed across multiple sources with varying structures and formats.
- Inconsistent job titles, skills, locations, and company information reduce data comparability.
- Missing, duplicate, and outdated records affect data quality.
- Multilingual job postings create additional challenges in data standardization.
- The lack of centralized, structured data makes it difficult to efficiently identify market trends and in-demand skills.

2.3 Impact

These challenges increase the effort required to collect and analyze job market data and can lead to incomplete or unreliable insights. Without a consistent and up-to-date data foundation, organizations and researchers may face difficulties in accurately understanding technology employment trends and emerging skill demands.

3. PROPOSED SOLUTION

3.1 SOLUTION OVERVIEW

The proposed solution is a single, automated data pipeline that solves the fragmentation problem described in Section 2. It runs on a schedule, applies the same rules every time, and only lets validated records into the final dataset — making it automated, repeatable, and reliable.

3.2 WHAT WE ARE BUILDING

The core of the solution is one pipeline that automatically connects to five selected sources — Jooble, JSearch, FreeHire, Tapneo, and Tanqeeb — collecting new postings on an ongoing basis rather than as a one-time effort. From the start, it is built to handle the real-world issues already observed in the raw data: sources structure their postings differently, some records are missing key fields, the same job often appears on more than one platform, and content can be in Arabic or English. Handling these issues is part of the pipeline's core design, not an afterthought.

3.3 WHAT IT DELIVERS

As data moves through the pipeline, it passes through three layers: Bronze for raw data, Silver for cleaned and standardized records, and Gold for the final, merged dataset. Along the way, postings are cleaned and mapped into one common structure, duplicate listings are detected and removed, and every record is validated before it is accepted. This entire process runs automatically on a schedule, with no manual intervention, and produces one reliable, structured, analysis-ready dataset — a solid foundation for future dashboards or market analysis.

4. TARGET STAKEHOLDERS

4.1 PRIMARY USERS: JOB SEEKERS & TECH TALENT

Fresh graduates and professionals seeking opportunities in the Saudi market. The platform provides them with actionable insights into in-demand technical skills, realistic salary expectations, trending job roles, and hiring patterns across different regions

4.2 SECONDARY USERS: TECH RECRUITERS & HR PROFESSIONALS

Hiring teams and recruiters looking to optimize their talent acquisition strategies. The platform enables them to benchmark role requirements, track market competitiveness, and identify skill availability within the local tech ecosystem

4.3 REGIONAL & LABOR MARKET ANALYSTS

Data analysts, government researchers, and regional planners who evaluate regional workforce distribution. The platform enables them to analyze geographic hiring concentrations, identify localized talent shortages across Saudi provinces (e.g., Central, Eastern, Western), and evaluate the impact of local employment initiatives.

5. TIMELINE

5.1 WEEKS THREE & FOUR: EXTRACTION & CLEANING (SEP 12–21)

The team builds and automates extraction scripts (Python + requests, Azure Functions) for Jooble, Tanqeeb, and RapidAPI, producing raw files in the Bronze layer every six hours. Each file is then cleaned with Pandas and automated as well, uploading cleaned files to the Silver layer.

5.2 WEEKS FIVE & SIX: MERGING, QUALITY & DELIVERY (SEP 22–30)

The three Silver files are merged using Pandas and RapidFuzz to remove duplicates and check quality, producing one final file in the Gold layer. The team then tests the full pipeline end-to-end, completes documentation, and prepares the final presentation.

6. DATA SOURCES

The project relies on five selected sources to ensure comprehensive coverage of the technology job market in Saudi Arabia.

The sources represent different connectivity types:

- **Jooble API:** Official API source. Connectivity is established via HTTPS POST request to https://jooble.org/api/{API_KEY} with a JSON body containing keywords and location. Authentication is via API key.

- **JSearch API (RapidAPI):** Managed REST API source. Connectivity is via HTTPS GET request to the RapidAPI gateway. Authentication requires X-RapidAPI-Key and X-RapidAPI-Host headers. Pagination is cursor-based, using a next_page_cursor value returned with each response to retrieve subsequent pages.

- **FreeHire API:** Browser-simulated API source. Connectivity is via HTTPS GET request to an internal search endpoint (https://freehire.me/api/v1/jobs/search), authenticated implicitly through browser-context headers (User-Agent, Referer) rather than an API key. Pagination is handled via limit and offset parameters. This source natively supports technology-job filtering through an is_tech parameter.

- **TapNeo Dataset:** File-based source. Connectivity is via direct file download in CSV/JSON format. It serves as a historical baseline and does not require continuous API connectivity.

- **Tanqeeb Aggregator:** Web-based source with no official API. Connectivity is via web scraping, routed through the ScrapeOps Proxy API to bypass the source's bot-detection protections. HTML content is parsed using Python libraries once retrieved.

6. DATA SOURCES

6.1 SOURCE CONNECTIVITY AND INGESTION STRATEGY

Given the heterogeneity of the five data sources, each requiring a distinct connectivity mechanism, the pipeline was designed around a unified batch-ingestion strategy rather than real-time streaming. This section outlines the connectivity patterns, pagination handling, and resilience mechanisms applied across all sources.

1. Connectivity Patterns

Sources are grouped into three connectivity categories, each requiring a different authentication and request-handling approach:

- Key-based API authentication (Jooble, JSearch): Requests are authenticated via a static API key, either embedded directly in the request URL (Jooble) or passed through dedicated headers (X-RapidAPI-Key, X-RapidAPI-Host for JSearch).

- Browser-context authentication (FreeHire): Rather than a formal API key, connectivity relies on replicating browser request headers (User-Agent, Referer) to establish a valid session with the source's internal search endpoint.

- Proxy-mediated scraping (Tanqeeb): Requests are routed through the ScrapeOps Proxy API, which handles IP rotation and header simulation to bypass the source's bot-detection layer, returning the underlying page content for parsing.

- Static file retrieval (TapNeo): No live connectivity is required; the dataset is downloaded once and read directly from local storage.

2. Ingestion Pattern

The pipeline follows a pull-based batch ingestion model: each source is queried on demand or on a scheduled basis, rather than receiving continuous data pushes. This approach was selected because:

- All sources expose request/response APIs (or scrapeable pages) rather than event streams or webhooks.
- Job postings are inherently low-frequency data (new listings appear over hours or days, not seconds), making batch pulls sufficient to capture new records without unnecessary API consumption.
- Batch ingestion aligns naturally with the Bronze-Silver-Gold layered storage model, where each run produces a traceable, timestamped raw snapshot.

3. Pagination Handling

Each API-based source implements a different pagination mechanism, requiring source-specific extraction logic:

Source	Pagination Type	Mechanism
Jooble	Page-based	Sequential `page` parameter, incremented until an empty result set is returned
JSearch	Cursor-based	A `next_page_cursor` token returned with each response is passed into the subsequent request
FreeHire	Offset-based	`limit` and `offset` parameters control the result window, incremented per iteration
Tanqeeb	Page-based (scraped)	Sequential page numbers embedded in the scraped URL structure
TapNeo	Not applicable	Delivered as a single bulk file; the entire dataset is retrieved in one download, with no request-level pagination required

Figure: Table 1 — Data Sources Mechanism

4. Rate Limiting and Quota Management

Given that most sources enforce request quotas (e.g., Jooble's monthly limit, RapidAPI's per-second and per-month limits), the ingestion logic incorporates the following safeguards:

- HTTP 429 Too Many Requests responses trigger a short backoff delay followed by a single retry attempt.
- Keyword and location combinations are deliberately scoped and estimated before execution, allowing the total projected request count to be validated against the remaining quota prior to a full run.
- A short inter-request delay is applied between paginated calls to avoid triggering aggressive rate limits.

5. Error Handling and Resilience

All extraction functions follow a consistent error-handling pattern:

- HTTP status codes are checked explicitly; non-200 responses are logged with the failing query and response body for diagnosis, rather than causing a silent failure or full pipeline crash.
- API-level error payloads (e.g., Jooble's and JSearch's structured status/error fields) are parsed and surfaced distinctly from transport-level (HTTP) errors.
- Each source module operates independently, so a failure in one source does not prevent the remaining sources from being ingested during the same run.

6. Landing Strategy

Regardless of connectivity method, all ingested data — whether from a live API call, a scraped page, or a static file — is landed unmodified into the Bronze layer, preserving a faithful, source-specific raw copy prior to any cleaning or standardization. This ensures that connectivity or parsing issues discovered later can be diagnosed against the original payload, without requiring the source to be re-queried.

Data Sources Table

To provide a clear overview of how each data source was accessed and integrated into the pipeline, Table 1 below summarizes the five sources used in this project, along with their access method, data format, and original reference link. This summary serves as a quick-reference guide before diving into the detailed connectivity and ingestion strategy discussed in the following section.

Source	Access	Format	Source link
Jooble	Official API (POST request with a custom key)	JSON	https://jooble.org/api/
JSearch	API via RapidAPI (GET request)	CSV / JSON	rapidapi.com
FreeHire	API Request via Browser	JSON	https://freehire.me/
Tapneo	Static Dataset Download + Pandas Read	CSV/JSON file	tapneo-data.com
Tanqeep	Web Scraping (Via ScrapeOps Proxy API)	JSON (Semi-Structured Data)	https://saudi.tanqeeb.com/ar

Figure: Table 2 — Data Sources Summary

Sample Output: TapNeo Dataset Preview

Figure 2 presents a preview of the cleaned and filtered TapNeo dataset, generated from the LinkedIn Saudi Jobs dataset after applying technology-role filtering based on job function and industry classification. The table displays the first ten records, showing four key fields — job title, company name, location, and posting date — providing a representative sample of the structured output produced by this source prior to aggregation with the remaining four sources.

	title	company	location	date
0	AI Engineer - GenAI	Hiredge Solutions	Riyadh	2026-04-18
1	HungerStation is hiring a Senior Data Engineer...	HungerStation	Riyadh	2026-04-13
2	We are looking for a skilled Data Analyst to j...	Assystem	Riyadh	2026-04-15
3	Position Overview We are looking for a Data En...	Master Works	Riyadh	2026-03-01
4	A Data Engineer is expected to develop monitor...	Devoteam	Riyadh	2026-04-05
5	Mllers Solutions is seeking a talented Data En...	Mllers Solut	Riyadh	2026-02-23
6	Riyadh Air RX headquartered in the Saudi Capit...	Riyadh Air	Riyadh	2026-03-31
7	Senior Data Engineer - AI	Blomal	Jeddah	2026-04-13
8	The Apprenticeship covers all domains of Syste...	Airbus Defence and Space	Riyadh	2026-04-18
9	We are seeking a datadriven professional to jo...	MCG Talent	Riyadh	2026-04-12

Figure: tapneo_table_preview.png

7. IMPLEMENTATION PLAN

The proposed Implementation Plan is designed to collect job postings from multiple sources and transform fragmented raw data into a standardized, validated, and structured dataset. The architecture separates raw data storage from data processing and final storage to preserve the original records while allowing the processed data to be continuously updated.

The pipeline will be implemented primarily using Python and Azure services. Python will be responsible for data extraction, processing, transformation, validation, and loading, while Azure Data Lake Storage Gen2 will be used to preserve raw data and used to store the final curated dataset. Azure Functions will provide the automation layer that triggers the pipeline periodically.

The proposed Implementation Plan consists of main stages: **data collection, raw data storage, transformation and cleaning, deduplication and data quality validation, curated data storage, and automation.**

7.1 DATA COLLECTION

The first stage of the pipeline is responsible for collecting job postings from multiple job-market sources. The project will initially use Jooble API, Tanqeeb Aggregator, RapidAPI JSearch , TapNeo and Freehire as the selected sources, subject to availability and access constraints.

Using multiple sources allows the project to collect a broader range of job postings and reduces dependence on a single data provider. However, using multiple sources also introduces differences in data structure, field names, formatting, and completeness. These differences will be addressed during the transformation stage.

Python will be used as the primary programming language for the extraction process. The requests library will be used to send requests to the available APIs and retrieve job-posting data.

The extracted records will initially remain in their source format to preserve the original information.

7.2 RAW DATA STORAGE

Once the data has been extracted, the original records will be stored in Azure Data Lake Storage Gen2. Python's azure-storage-file-datalake library will be used to upload the collected JSON files directly into the Data Lake file system.

The raw layer is intentionally kept separate from the processed dataset. This allows to maintain an original copy of the collected information before applying transformations or cleaning rules , therefore allows the team to reproduce or modify the transformation process without needing to collect the original data again.

7.3 TRANSFORMATION AND CLEANING

After the raw data has been stored, the pipeline will read the files from Azure Data Lake Storage Gen2 and process them using Python and Pandas.

The main objective of this stage is to convert data from different sources into a common and consistent structure , Because each source may use different field names, the pipeline will create a Common Schema.

Transformations may include :

- Renaming columns.
- Converting data types.
- Standardizing date formats.
- Cleaning text fields.
- Handling missing values.
- Standardizing employment types.
- Normalizing location values.
- Extracting relevant information from descriptions.
- Removing unnecessary fields.

Pandas will be used because it provides suitable functionality for manipulating tabular data and applying repeatable transformation rules.

7.4 DEDUPLICATION

A major challenge associated with combining multiple job sources is the presence of duplicate job postings. The same job vacancy may appear on multiple platforms, even though each source assigns it a different identifier or represents the information slightly differently.Therefore, the pipeline will include a dedicated deduplication step .

RapidFuzz will be used to support fuzzy matching between records, The deduplication process will calculate similarity between relevant fields to identify likely duplicate records.

The objective is to reduce duplicate job postings while avoiding the removal of genuinely different vacancies.

7.5 DATA QUALITY AND VALIDATION

Following transformation and deduplication, the pipeline will perform data-quality checks using Python and Pandas.

The validation stage is intended to ensure that only records meeting the project's minimum quality requirements are included in the final dataset.

The initial validation rules will include checking the presence of required fields , The pipeline will also check for :

- Missing required values.
- Invalid or unexpected data types.
- Empty records.
- Duplicate records remaining after deduplication.
- Invalid or malformed URLs where applicable.
- Inconsistent values in standardized fields.

Records that fail critical validation checks can either be excluded from the curated dataset or flagged for further investigation, depending on the type and severity of the issue.

7.6 CURATED DATASET

Once the data has passed through the transformation, cleaning, deduplication, and validation stages, the resulting records will be stored as the project's Curated Dataset.

The curated data will be loaded into Azure Data Lake Storage Gen2 to serve as the Silver Layer. This keeps the cleaned dataset separate from the raw files and allows for efficient storage and querying.

All records in this layer follow a unified Common Schema, ensuring consistent column names, cleaned text, normalized Saudi city locations, and verified technical skills with all duplicates removed.

The curated dataset serves as the main output of the data engineering pipeline. It provides a reliable and structured foundation that can directly support analytical queries, reporting, dashboards, or other downstream applications.

7.7 AUTOMATION

To make the data pipeline repeatable and reduce manual intervention, Azure Functions will be used as the automation layer. Instead of manually executing each Python script, the Azure Function will trigger the pipeline according to a predefined schedule.

The automation layer will not be limited to data extraction. Instead, it will trigger the complete workflow, including data collection, raw data storage, transformation and cleaning, deduplication, data-quality validation, and loading of the final curated dataset into Azure Data Lake Storage Gen2.

This approach allows newly available job postings to be incorporated into the dataset periodically while ensuring that the same processing and validation rules are applied consistently during each pipeline execution.

7.8 TECHNOLOGIES USED

Technology	Role in the Pipeline
Python	Main programming language for Extract, Transform, Validate, and Load operations
Requests / BeautifulSoup	Retrieving job data from web sources and APIs
Azure Data Lake Storage Gen2	Scalable cloud storage for both raw data (Bronze) and curated datasets (Silver)
Pandas	Data cleaning, transformation, and standardization
RapidFuzz	Fuzzy matching and duplicate detection
azure-storage-file-datalake	Interacting with and managing files within ADLS Gen2 programmatically
Azure Functions	Automating and scheduling the pipeline
GitHub	Source-code management and version control

7.9 DOWNSTREAM APPLICATIONS

The curated dataset can be used for optional downstream applications if time and project scope allow. These may include interactive dashboard or job-market analysis.

The dashboard could provide visual insights into the collected job-market data, such as the distribution of job postings by location, job title, company, skills, education requirements, and employment type. This extension would build on the curated dataset produced by the core data engineering pipeline.

8. EXPECTED DELIVERABLES

8.1 Technical Deliverables

- **Unified Data Pipeline:** A modular, source-agnostic ingestion pipeline capable of extracting technology job postings from five heterogeneous sources (Jooble, JSearch, FreeHire, TapNeo, Tanqeeb), each handled through its appropriate connectivity method (API, browser-simulated request, proxy-mediated scraping, or static file retrieval).

- **Layered Data Lake Structure:** A structured Bronze-Silver-Gold storage architecture within Azure Data Lake Storage Gen2, ensuring raw data traceability (Bronze), standardized and deduplicated records (Silver), and analysis-ready datasets (Gold) in CSV/Parquet format.

- **Standardized Job Schema:** A unified data model that normalizes inconsistent fields across sources (e.g., job title, company, location, posting date) into a single, queryable schema, resolving source-specific formatting differences.

- **Deduplication and Data Quality Logic:** Automated fuzzy-matching deduplication (via Rapidfuzz) and geographic validation logic to eliminate duplicate postings and filter out mislocated or non-Saudi entries with misleading location matches.

- **Reusable Extraction Modules:** Independently testable Python modules per source, allowing new data sources to be onboarded without modifying the core pipeline logic.

8. EXPECTED DELIVERABLES

8.2 Business Deliverables

- **Skill Demand Insights for Job Seekers:** An aggregated view of in-demand technical roles and skills across the Saudi market, enabling fresh graduates and professionals to align their career preparation with actual hiring trends rather than anecdotal assumptions.

- **Salary and Role Benchmarking for Recruiters:** Consolidated role-requirement and market-competitiveness data, allowing tech recruiters and HR professionals to benchmark job postings against real-time market activity rather than outdated internal references.

- **Curriculum Alignment Insights for Academic Institutions:** Aggregated skill-gap and emerging-technology signals, enabling bootcamps and academic institutions to adjust training programs based on verified market demand instead of assumed industry needs.

- **Regional Labor Market Visibility for Analysts and Policymakers:** A geographically segmented view of hiring activity across Saudi provinces (Central, Eastern, Western), supporting evidence-based analysis of regional talent shortages and the effectiveness of localized employment initiatives.

- **A Single Source of Truth for the Saudi Tech Job Market:** By consolidating five fragmented sources into one standardized dataset, the project eliminates the need for stakeholders to manually cross-reference multiple platforms, reducing research overhead across all four user groups.